

Failure handling (three errors, three reactions) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — three separate `except` blocks, each reacting differently

```
# custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type):
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        print("Will retry later, telling the warehouse: " + str(e))
    except PaymentDeclinedError as e:
        print("Asking for a different payment method: " + str(e))
    except InvalidAddressError as e:
        print("Asking the customer to fix their address: " + str(e))
```

Expected output:

```
Will retry later, telling the warehouse: item is out of stock
Asking for a different payment method: payment was declined
Asking the customer to fix their address: shipping address is invalid
```

This is correct and is exactly what the exercise asks for: 3 error types, each with its own reaction. It's missing type hints, which is fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — three separate, differently-reacting `except` blocks

```
# custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        print(f"Will retry later, notifying the warehouse: {e}")
    except PaymentDeclinedError as e:
        print(f"Asking the customer for a different payment method: {e}")
    except InvalidAddressError as e:
        print(f"Asking the customer to fix their address: {e}")
```

Expected output:

```
Will retry later, notifying the warehouse: item is out of stock
Asking the customer for a different payment method: payment was declined
```

Asking the customer to fix their address: shipping address is invalid

Approach 2 — a shared base class, for when some handling really is shared

```
# custom_errors_practice.py – Failure handling section
class OrderError(Exception):
    pass

class OutOfStockError(OrderError):
    pass

class PaymentDeclinedError(OrderError):
    pass

class InvalidAddressError(OrderError):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except PaymentDeclinedError as e:
        print(f"Asking the customer for a different payment method: {e}")
    except OrderError as e:
        # catches OutOfStockError and InvalidAddressError together
        print(f"Order problem, logging and notifying support: {e}")
```

Expected output:

```
Order problem, logging and notifying support: item is out of stock
Asking the customer for a different payment method: payment was declined
Order problem, logging and notifying support: shipping address is invalid
```

Difference from Basic: both Intermediate approaches add full type hints (`problem_type: str, - > None`). The real difference is between the two approaches: Approach 1 reacts to all 3 errors independently. Approach 2 makes `OutOfStockError` and `InvalidAddressError` inherit from a shared `OrderError` base and handles them together — notice the output changes for the stock and address cases, because they now hit the shared `except OrderError:` branch instead of their own. `PaymentDeclinedError` is still caught on its own, since Python checks `except` clauses top to bottom and stops at the first match.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — separate handler functions, and errors that carry structured data

```
# custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    def __init__(self, item_id: str) -> None:
        self.item_id = item_id
        super().__init__(f"item {item_id} is out of stock")

class PaymentDeclinedError(Exception):
    def __init__(self, reason: str) -> None:
        self.reason = reason
        super().__init__(f"payment declined: {reason}")

class InvalidAddressError(Exception):
    def __init__(self, address: str) -> None:
        self.address = address
        super().__init__(f"shipping address is invalid: {address}")

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("SKU-42")
    if problem_type == "payment":
        raise PaymentDeclinedError("insufficient funds")
    if problem_type == "address":
        raise InvalidAddressError("123 Nowhere St")

def handle_out_of_stock(e: OutOfStockError) -> None:
    print(f"Retrying later, notifying warehouse about {e.item_id}")

def handle_payment_declined(e: PaymentDeclinedError) -> None:
    print(f"Asking customer for a new payment method ({e.reason})")

def handle_invalid_address(e: InvalidAddressError) -> None:
```

```

    print(f"Asking customer to fix address: {e.address}")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        handle_out_of_stock(e)
    except PaymentDeclinedError as e:
        handle_payment_declined(e)
    except InvalidAddressError as e:
        handle_invalid_address(e)

```

Expected output:

```

Retrying later, notifying warehouse about SKU-42
Asking customer for a new payment method (insufficient funds)
Asking customer to fix address: 123 Nowhere St

```

Each exception now carries the actual data a handler needs (`e.item_id` , `e.reason` , `e.address`) instead of only a printable message, and the reaction logic lives in its own named function rather than inline in the `except` block — easier to test each reaction on its own.

Approach 2 — one shared base class, with the reaction chosen by data, not by type

```

# custom_errors_practice.py – Failure handling section
class OrderError(Exception):
    def __init__(self, message: str, *, recoverable: bool) -> None:
        self.recoverable = recoverable
        super().__init__(message)

class OutOfStockError(OrderError):
    def __init__(self, item_id: str) -> None:
        self.item_id = item_id
        super().__init__(f"item {item_id} is out of stock", recoverable=True)

class PaymentDeclinedError(OrderError):
    def __init__(self, reason: str) -> None:
        self.reason = reason
        super().__init__(f"payment declined: {reason}", recoverable=True)

class InvalidAddressError(OrderError):

```

```

def __init__(self, address: str) -> None:
    self.address = address
    super().__init__(f"shipping address is invalid: {address}", recoverable=F

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("SKU-42")
    if problem_type == "payment":
        raise PaymentDeclinedError("insufficient funds")
    if problem_type == "address":
        raise InvalidAddressError("123 Nowhere St")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OrderError as e:
        if e.recoverable:
            print(f"Recoverable, will retry: {e}")
        else:
            print(f"Not recoverable, needs a human: {e}")

```

Expected output:

```

Recoverable, will retry: item SKU-42 is out of stock
Recoverable, will retry: payment declined: insufficient funds
Not recoverable, needs a human: shipping address is invalid: 123 Nowhere St

```

There's only **one** `except` clause here (`except OrderError as e:`), for all 3 error types. The reaction is chosen by reading a piece of data on the exception (`e.recoverable`), not by which `except` block matched. This is a different kind of dispatch than Intermediate's Approach 2 — that one still chose the reaction by *type* (a specific `except PaymentDeclinedError` plus a catch-all), this one chooses by a *value* carried on the object.

Approach 3 — a dispatch table, for many error types

```

# custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

```

```

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

def handle_stock(e: Exception) -> None:
    print(f"Will retry later, notifying the warehouse: {e}")

def handle_payment(e: Exception) -> None:
    print(f"Asking the customer for a different payment method: {e}")

def handle_address(e: Exception) -> None:
    print(f"Asking the customer to fix their address: {e}")

HANDLERS = {
    OutOfStockError: handle_stock,
    PaymentDeclinedError: handle_payment,
    InvalidAddressError: handle_address,
}

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except Exception as e:
        handler = HANDLERS.get(type(e))
        if handler is not None:
            handler(e)
        else:
            print(f"Unknown error: {e}")

```

Expected output:

```

Will retry later, notifying the warehouse: item is out of stock
Asking the customer for a different payment method: payment was declined
Asking the customer to fix their address: shipping address is invalid

```

There is exactly one `except` clause in the whole program (`except Exception as e:`), and it never needs to change. Adding a 4th error type — say, `PaymentTimeoutError` — means writing one new handler function and adding one line to `HANDLERS` . No existing code is touched, and there's no growing `except` chain to scroll past.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's approaches both dispatch by writing one `except ExceptionType:` per case (Approach 2 groups some of them under a shared base, but it's still a fixed chain of `except` clauses in the source code). All 3 Advanced approaches move data *onto* the exception object so the handling logic has more to work with, but they scale differently. Approach 1 keeps 3 separate `except` blocks, but pulls each reaction into its own function and gives each exception real attributes — a good next step when you still only have a handful of error types. Approach 2 collapses to a single `except OrderError:` and chooses the reaction from an attribute (`recoverable`) — great when many error types genuinely fall into a small number of *reaction categories*. Approach 3 collapses to a single `except Exception:` and a lookup table keyed by the exact class — best when you have (or expect to grow into) many distinct error types that each need their own, different, individually-testable reaction.

Which one should you actually write? For this exercise, Intermediate Approach 1 (three separate `except` blocks, each reacting differently) is what you should write — it's exactly what's being tested, and it's the clearest code for 3 error types. Intermediate Approach 2's shared base class is worth it the moment two or more error types really do deserve the same reaction. Reach for Advanced only once the *number of error types* becomes the actual problem: Approach 1 when you want structured data on 3–5 errors handled individually; Approach 2 when many errors collapse into a couple of shared reaction categories; Approach 3 when you're handling many distinct error types (dozens, or a growing list) and want adding a new one to mean "one dict entry," not "one more `except` block in an ever-longer chain."